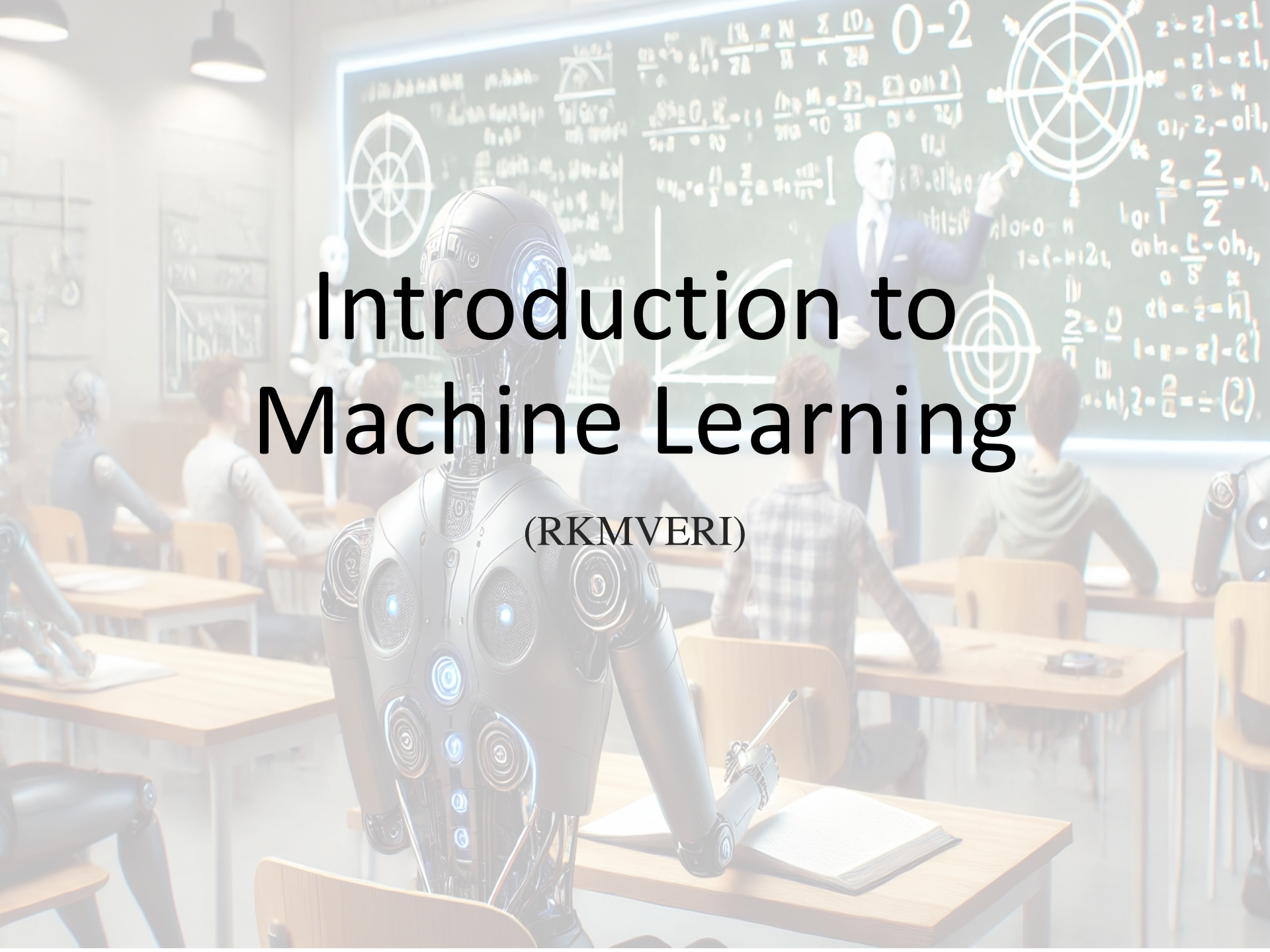




Introduction to Machine Learning

(RKMVERI)

Shanti Mantra

ॐ सह नाववतु ।

सह नौ भुनक्तु ।

सह वीर्यं करवावहै ।

तेजस्वि नावधीतमस्तु मा विद्विषावहै ।

ॐ शान्तिः शान्तिः शान्तिः ॥

Om Saha Nau-Avatu |

Saha Nau Bhunaktu |

Saha Viiryam Karavaavahai |

Tejasvi Nau-Adhiitam-Astu Maa Vidvissaavahai |

Om Shaantih Shaantih Shaantih ||

Om, may He protect us both (the teacher and the student).

May He nourish us both.

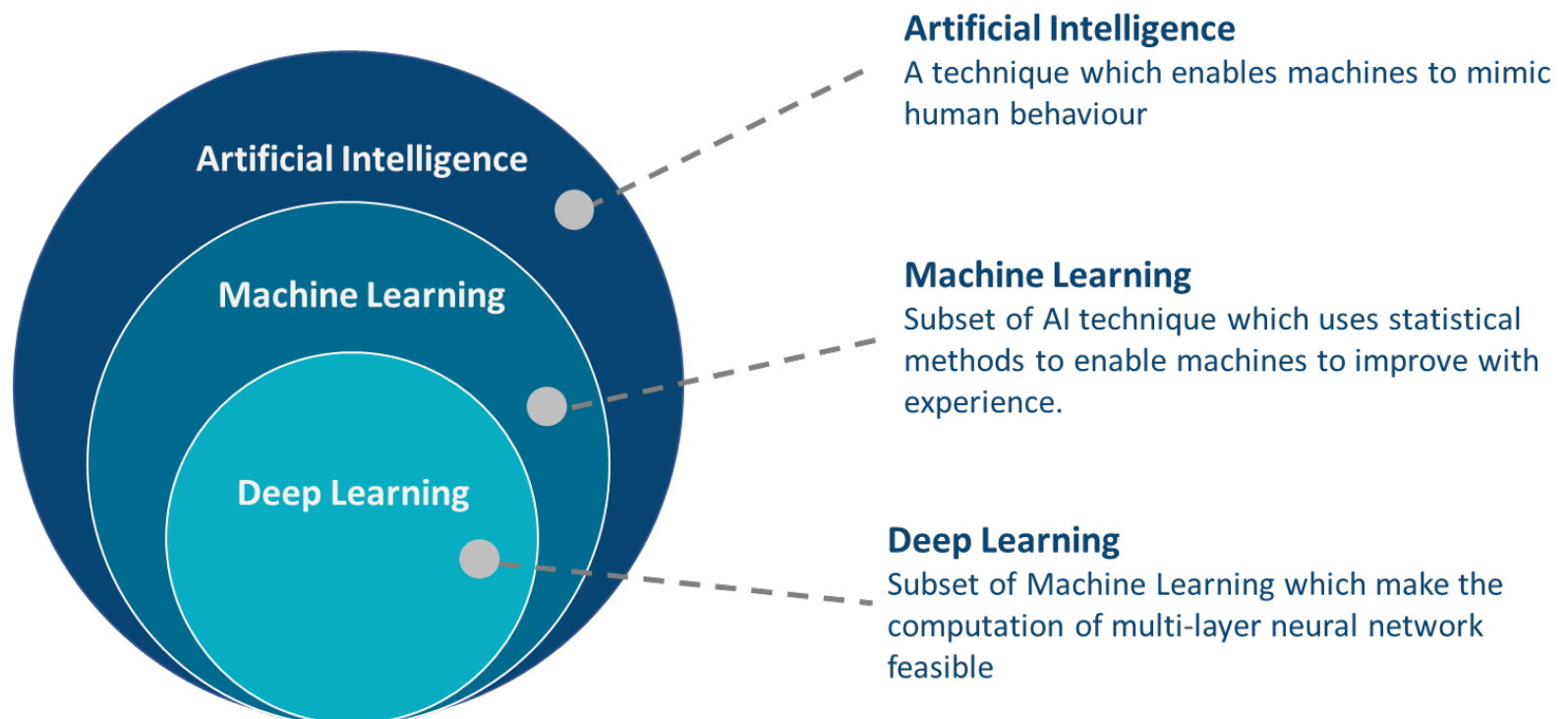
May we work together with energy and vigor.

May our study be enlightening and not give rise to hostility.

Om, peace, peace, peace.

What is machine learning?

- Machine learning (ML) is a branch of artificial intelligence that enables systems to learn from data and make decisions or predictions without being explicitly programmed.



Machine Learning definition

- Arthur Samuel (1959). Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) Well-posed Learning Problem:
- A computer program is said to *learn* from **experience E** with respect to some **task T** and some performance **measure P**, if its performance on T, as measured by P, improves with experience E.

How to divide the data between X-train and X-test?

- The data is split into two main parts:
- **Training set (X-train, Y-train):** Used to train the model. Typically, 70-80% of the dataset is used for this.
- **Test set (X-test, Y-test):** Used to evaluate the model's performance on unseen data, usually 20-30% of the dataset.
- In Python, you can use `train_test_split()` from **Scikit-Learn**:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

How to process data?

- Data processing involves steps to prepare data for analysis, such as:
- **Data cleaning** (handling missing values, removing duplicates)
- **Feature extraction/selection** (choosing relevant features)
- **Encoding categorical variables** (converting text labels to numbers)
- **Scaling or normalization** (ensuring all features are on the same scale)
- Tools like Pandas and Scikit-Learn offer functions for these tasks

How to clean up data?

- Data cleaning involves:
- **Removing duplicates**
 - `df_no_duplicates = df.drop_duplicates()`
- **Handling missing values** (e.g., removing rows with NaN, filling missing values with mean/median/mode)
 - `df_cleaned = df.dropna()`
 - `df.fillna(df.mean(), inplace=True)`
 - `df['age'].fillna(df['age'].mean(), inplace=True)`
- **Handling outliers**
 - `df_filtered = df[df['age'] >= 30]`
- **Correcting data types** (e.g., ensuring numerical features are treated as floats/integers)
 - `df['salary'] = df['salary'].astype(float)`

How to standardize data?

- Standardization involves scaling features so that they have a mean of 0 and a standard deviation of 1. This is especially important for algorithms like gradient descent. In Scikit-Learn, you can use:

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```


How to skip NaN?

- To skip or handle missing values (NaN), you can:
- **Drop rows or columns** with missing values using `dropna()`.
- **Fill missing values** with mean, median, or a constant using `fillna()`.

```
df.dropna() # Remove rows with NaN
```

```
df.fillna(df.mean(), inplace=True) # Replace NaN with column mean
```

Repeat SoftMax Regression

- Softmax regression is an extension of **logistic regression** used for multi-class classification.
- It computes the probability of each class using the **softmax function** and chooses the class with the highest probability.
- Please meet me in person if you have further doubt!

SoftMax Regression

Probability distribution:

$$p_k^{(i)} = P(y^{(i)} = k \mid x^{(i)}) = \frac{\exp(\theta_k^T x^{(i)})}{\sum_{j=1}^K \exp(\theta_j^T x^{(i)})}$$

Loss Function
(Cross-Entropy Loss):

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$



Gradient of the Loss Function:

$$\nabla_{\theta}^{(k)} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

How to work on multiple features for Simple LR?

- In simple linear regression, if you have multiple features (independent variables), this becomes **multiple linear regression**. You pass multiple columns as input features:

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
model.fit(X_train, y_train) # X_train can have multiple features
```

Polynomial Regression

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
# Example data (you would replace this with your actual dataset)
X = np.array([[2, 3], [3, 5], [4, 7], [5, 9]]) # Multiple features
y = np.array([15, 25, 35, 45]) # Target variable
```

```
poly = PolynomialFeatures(degree=2) # Choose the degree of the polynomial
X_poly = poly.fit_transform(X) # Generate polynomial and interaction terms
```

```
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2,
random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train) # Fit the model on the polynomial features
```

Custom Function for polynomial terms (degree=2)

```
def add_polynomial_terms(X):
    n_samples, n_features = X.shape
    X_poly = np.ones((n_samples, 1)) # Start with a column of ones (bias term)

    # Add the original features
    X_poly = np.hstack((X_poly, X))

    # Add squared terms and interaction terms for each feature
    for i in range(n_features):
        for j in range(i, n_features): # Start from i to avoid duplicates (e.g., x1*x2 == x2*x1)
            X_poly = np.hstack((X_poly, (X[:, i] * X[:, j]).reshape(-1, 1))) # Add
interaction/squared terms

    return X_poly
```

How to import dataset for ML assignment?

- You can import datasets using Pandas for most ML tasks:

```
import pandas as pd
df = pd.read_csv('data.csv') # For CSV files
```

- You can also use built-in datasets from Scikit-Learn:

```
from sklearn.datasets import load_iris
data = load_iris()
X, y = data.data, data.target
```


What is Scikit-Learn?

- **Scikit-learn** is
‘Scientific Kit (toolkit) Learn (Machine Learning)’
- Scikit-Learn is a popular Python library for machine learning.
- It provides simple and efficient tools for data mining, data analysis, and machine learning.
- It includes models for classification, regression, clustering, and dimensionality reduction, along with preprocessing and model evaluation functions.

How to handle missing values in a dataset?

- There are several approaches to handle missing values for regression:
- **Removing missing values** (using `dropna()`).
- **Imputing missing values** (with mean, median, or mode using `fillna()`).

```
# Fill NaN values with specific values for each column
df.fillna({
    'name': 'Unknown',    # Fill NaN in 'name' column with 'Unknown'
    'age': df['age'].mean(), # Fill NaN in 'age' column with the mean of the column
    'salary': df['salary'].median() # Fill NaN in 'salary' column with the median
}, inplace=True)

print("DataFrame after filling NaN for each column with different values:")
print(df)
```

linear_model.LinearRegression VS KNeighborsRegression

- **Linear Regression** assumes a linear relationship between features and the target.
- **K-Neighbors Regression (KNN Regression)** predicts the target based on the average of k-nearest data points. It doesn't assume a specific relationship between features and target.
- Use **KNN Regression** when you have a non-linear relationship between features and target.
- Also, Linear Regression is model-based, whereas, K-Neighbors is an instance based ML model

Batch, mini-batch, and stochastic gradient descent (SGD)

- **Batch Gradient Descent:** The gradient is calculated over the entire dataset.
- **Mini-Batch Gradient Descent:** The gradient is calculated on small subsets (mini-batches) of the data.
- **Stochastic Gradient Descent (SGD):** The gradient is calculated on a single data point. It's computationally less expensive but more noisy compared to batch methods.

Repeated cross-validation and why prefer it over a single validation set?

- Repeated cross-validation involves running k-fold cross-validation multiple times (with different random splits).
- It gives a more reliable estimate of model performance and reduces the variance compared to a single validation set.

Regularization hyperparameter and if learning rate is hyperparameter?

- **Regularization hyperparameter** controls the amount of regularization applied to the model to avoid overfitting.
- In models like Ridge and Lasso, it penalizes large coefficients.
- **Learning rate** is also a hyperparameter, controlling the step size during gradient descent.
- Everything except the model parameters are hyperparameters.

How does the model know which value of the model parameters will give minimum MSE?

- We use the gradient information
- The model uses optimization techniques, like gradient descent, to iteratively adjust the parameters in the direction that reduces MSE.
- It calculates the gradient (slope) of the MSE with respect to each parameter and updates the parameters to move towards the minimum MSE.
- This process continues until convergence or until a stopping criterion is met.



THANK
YOU :)